PropGenius AI — Complete Project Plan

AI-Powered Real Estate Listing Generator + CRM

Version 1.0 | February 2026

TABLE OF CONTENTS

1. Executive Summary
2. Business Model & Revenue
3. Tech Stack
4. System Architecture
5. Database Schema
6. Design System
7. Directory Structure
8. Feature Breakdown
9. AI Integration Details
10. API Routes
11. Development Phases
12. Environment Variables
13. Deployment & DevOps
14. Testing Strategy
15. Security Considerations
16. Future Roadmap

---

1. EXECUTIVE SUMMARY

Problem Statement

Indian real estate agents face two critical problems:

1. **Listing Creation is Time-Consuming**: Agents spend 30-60 minutes writing each property listing description manually. With 10-20 listings per month, that's 5-20 hours wasted on repetitive writing.
2. **Lead Leakage**: 60-70% of leads are lost due to poor follow-up. Agents use WhatsApp and phone calls without any system to track, score, or prioritize leads.

Solution: PropGenius AI

A SaaS web application that:

- **AI Listing Generator**: Input property details, get professional listings in seconds (description, SEO title, social media posts, platform-specific formats)
- **Smart CRM**: Manage leads with AI-powered scoring, Kanban board, activity tracking, and automated WhatsApp follow-ups
- **Analytics Dashboard**: Track listing performance, lead conversion, and agent productivity

Target Market

- 500,000+ registered real estate agents in India
- 50,000+ small brokerages
- Growing PropTech market ($1.2B in India, 2025)

Competitive Advantage

- AI-first approach (not just a CRM with AI bolted on)
- Built for Indian market (₹ pricing, Indian property types, local platforms)
- WhatsApp-native communication (India's #1 business messenger)
- Free tier to drive adoption, premium for power features

---

2. BUSINESS MODEL & REVENUE

Subscription Plans (via Razorpay)

Plan	Price/Month	Annual (20% off)	Features
Free	₹0	₹0	5 AI listings/month, 50 leads, basic CRM, email support
Pro	₹999	₹9,588	Unlimited listings, 500 leads, WhatsApp integration, AI lead scoring, priority support
Business	₹2,499	₹23,988	Everything in Pro + team (5 agents), analytics dashboard, bulk operations, custom templates
Enterprise	Custom	Custom	Unlimited agents, API access, custom branding, dedicated account manager, SLA

Additional Revenue Streams

- **Pay-per-listing**: ₹49/listing for free users after quota
- **WhatsApp credits**: Extra message packs (₹199 for 500 messages)
- **Premium templates**: ₹299 for industry-specific listing templates
- **API access**: ₹4,999/month for integration with external property portals

Financial Projections (Year 1)

Month	Free Users	Pro	Business	MRR
Month 1	100	5	0	₹4,995
Month 3	500	25	3	₹32,472
Month 6	2,000	100	15	₹137,385
Month 12	5,000	300	50	₹424,500

Key Metrics to Track

- **MRR** (Monthly Recurring Revenue)
- **Churn Rate** (target: <5% monthly)
- **CAC** (Customer Acquisition Cost)
- **LTV** (Lifetime Value, target: >12x CAC)
- **Free-to-Pro Conversion** (target: 5-8%)
- **AI Listings Generated/Month**
- **Lead Conversion Rate**

---

3. TECH STACK

Frontend

Technology	Purpose
Next.js 16	App Router, Server Components, API Routes
TypeScript	Type safety across the entire codebase
Tailwind CSS 4	Utility-first styling with custom design tokens
React Hook Form	Form management with validation
Zod	Schema validation (forms + API)
TanStack React Query	Server state management, caching, mutations
TanStack Table	Data tables with sorting, filtering, pagination
Zustand	Lightweight client state (auth, sidebar, notifications)
Recharts	Analytics charts and visualizations
lucide-react	Icon library
@dnd-kit/core	Drag-and-drop for Kanban board

Backend

Technology	Purpose
Supabase	Auth, PostgreSQL database, Storage (images), Realtime subscriptions
Google Gemini 2.5 Flash	AI listing generation, lead scoring, insights
Next.js API Routes	Server-side logic, webhooks

Integrations

Service	Purpose
Razorpay	Subscription billing and payments
WhatsApp Business API	Customer messaging (via Twilio or Meta Cloud API)
Resend	Transactional emails (welcome, password reset, notifications)
Google Maps API	Location picker for property listings (optional)
Vercel	Deployment and hosting
GitHub Actions	CI/CD pipeline

---

4. SYSTEM ARCHITECTURE

High-Level Architecture

```
                    ┌──────────────────────┐
                    │    Users (Agents)    │
                    │  Browser / Mobile    │
                    └──────────┬───────────┘
                               │
                    ┌──────────▼───────────┐
                    │     Vercel Edge      │
                    │  (CDN + Middleware)  │
                    └──────────┬───────────┘
                               │
          ┌────────────────────┼────────────────────┐
          │                    │                    │
┌─────────▼─────────┐  ┌───────▼───────────┐  ┌─────────▼─────────┐
│  Next.js SSR      │  │  API Routes       │  │  Static Assets    │
│  Pages            │  │  /api/*           │  │  /public          │
```

```
│  (Dashboard,     │  │  (AI, Webhooks, │  │  (Images, Logo)  │
│   Listings,      │  │   CRUD)         │  │                  │
│   Leads)         │  │                 │  │                  │
└──────────────────┘  └─────────────────┘  └──────────────────┘
         │                   │
         │                   │
         │     ┌─────────────┼─────────────┐
         │     │             │             │
         ▼     ▼             ▼             ▼

┌──────────────────┐  ┌───────────────┐  ┌──────────────────┐
│    Supabase      │  │  Gemini AI    │  │  External APIs   │
│  ┌────────────┐  │  │               │  │  ┌────────────┐  │
│  │ Auth       │  │  │ - Generate    │  │  │ Razorpay   │  │
│  │ PostgreSQL │  │  │   Listings    │  │  │ WhatsApp   │  │
│  │ Storage    │  │  │ - Score       │  │  │ Resend     │  │
│  │ Realtime   │  │  │   Leads       │  │  │ Google Maps│  │
│  └────────────┘  │  │ - Insights    │  │  └────────────┘  │
└──────────────────┘  └───────────────┘  └──────────────────┘
```

Data Flow: AI Listing Generation

```
Agent fills form → POST /api/ai/generate-listing
                        │
                        ▼
              Validate with Zod schema
                        │
                        ▼
              Build Gemini AI prompt
              (property details + context)
                        │
                        ▼
              Gemini 2.5 Flash generates:
              - Title
              - Description (150-200 words)
              - 5 Key Highlights
              - Social Media Post
              - SEO Meta Description
                        │
                        ▼
              Return JSON to frontend
                        │
                        ▼
              Agent reviews + edits
                        │
                        ▼
```

```
                Save to Supabase (listings table)
                Upload images to Supabase Storage
```

Data Flow: Lead Scoring

```
New lead created/updated → POST /api/ai/score-lead
                                    |
                                    ▼
                            Collect lead data:
                            - Budget vs listing price
                            - Response time
                            - Source quality
                            - Engagement level
                            - Number of inquiries
                                    |
                                    ▼
                            Gemini AI analyzes and returns:
                            - Score (0-100)
                            - Reason (text explanation)
                            - Recommended action
                                    |
                                    ▼
                            Update lead record in Supabase
                            Trigger notification if hot lead (>80)
```

---

5. DATABASE SCHEMA

Entity Relationship Diagram

```
organizations ──┬─< profiles
                ├─< listings
                ├─< leads ──────< lead_activities
                ├─< whatsapp_messages
                └─< subscriptions

leads ──< whatsapp_messages
leads ──< lead_activities
listings ──< leads (interested_in)
profiles ──< listings (created_by)
```

```
profiles ──< leads (assigned_to)
profiles ──< lead_activities (performed_by)
```

Table Definitions

organizations

```sql
CREATE TABLE organizations (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name TEXT NOT NULL,
  slug TEXT UNIQUE NOT NULL,
  logo_url TEXT,
  address TEXT,
  city TEXT,
  phone TEXT,
  email TEXT,
  plan TEXT NOT NULL DEFAULT 'free'
    CHECK (plan IN ('free', 'pro', 'business', 'enterprise')),
  razorpay_customer_id TEXT,
  razorpay_subscription_id TEXT,
  max_listings INT DEFAULT 5,
  max_leads INT DEFAULT 50,
  max_agents INT DEFAULT 1,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

profiles

```sql
CREATE TABLE profiles (
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,
  organization_id UUID REFERENCES organizations(id) ON DELETE CASCADE,
  full_name TEXT NOT NULL,
  email TEXT NOT NULL,
  phone TEXT,
  avatar_url TEXT,
  role TEXT NOT NULL DEFAULT 'owner'
    CHECK (role IN ('owner', 'admin', 'agent')),
  monthly_listing_count INT DEFAULT 0,
  is_active BOOLEAN DEFAULT TRUE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
```

```
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

listings

```
CREATE TABLE listings (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  created_by UUID NOT NULL REFERENCES profiles(id),
  -- Property Details
  title TEXT NOT NULL,
  description TEXT,
  property_type TEXT NOT NULL
    CHECK (property_type IN ('apartment', 'house', 'villa', 'plot', 'commercial', 'pg')),
  transaction_type TEXT NOT NULL
    CHECK (transaction_type IN ('sale', 'rent', 'lease')),
  price NUMERIC NOT NULL,
  price_unit TEXT DEFAULT 'total'
    CHECK (price_unit IN ('total', 'per_sqft', 'per_month')),
  -- Location
  address TEXT,
  locality TEXT,
  city TEXT NOT NULL,
  state TEXT NOT NULL,
  pincode TEXT,
  latitude NUMERIC,
  longitude NUMERIC,
  -- Specifications
  bedrooms INT,
  bathrooms INT,
  area_sqft NUMERIC,
  carpet_area_sqft NUMERIC,
  floor_number INT,
  total_floors INT,
  furnishing TEXT CHECK (furnishing IN ('unfurnished', 'semi', 'fully')),
  facing TEXT,
  age_years INT,
  parking INT DEFAULT 0,
  -- Amenities
  amenities JSONB DEFAULT '[]',
  -- Media
  images TEXT[] DEFAULT '{}',
  floor_plan_url TEXT,
  virtual_tour_url TEXT,
  -- AI Generated
  ai_description TEXT,
```

```
  ai_social_post TEXT,
  ai_seo_title TEXT,
  ai_seo_description TEXT,
  ai_highlights JSONB DEFAULT '[]',
  -- Status
  status TEXT DEFAULT 'draft'
    CHECK (status IN ('draft', 'active', 'sold', 'rented', 'archived')),
  views_count INT DEFAULT 0,
  inquiries_count INT DEFAULT 0,
  -- External
  published_platforms JSONB DEFAULT '[]',
  external_urls JSONB DEFAULT '{}',
  -- Timestamps
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

leads

```
CREATE TABLE leads (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  assigned_to UUID REFERENCES profiles(id),
  -- Contact Info
  name TEXT NOT NULL,
  email TEXT,
  phone TEXT,
  whatsapp_number TEXT,
  -- Lead Details
  source TEXT DEFAULT 'website'
    CHECK (source IN ('website', 'whatsapp', 'phone', 'walkin', 'referral',
                      '99acres', 'magicbricks', 'housing', 'other')),
  interested_in UUID REFERENCES listings(id) ON DELETE SET NULL,
  budget_min NUMERIC,
  budget_max NUMERIC,
  preferred_location TEXT,
  preferred_property_type TEXT,
  -- AI Scoring
  ai_score INT DEFAULT 0,
  ai_score_reason TEXT,
  ai_recommended_action TEXT,
  -- Status
  status TEXT DEFAULT 'new'
    CHECK (status IN ('new', 'contacted', 'interested', 'site_visit',
                      'negotiation', 'converted', 'lost')),
  lost_reason TEXT,
```

```
  -- Follow-up
  last_contacted_at TIMESTAMPTZ,
  next_followup_at TIMESTAMPTZ,
  followup_notes TEXT,
  -- Meta
  notes TEXT,
  tags TEXT[] DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

lead_activities

```
CREATE TABLE lead_activities (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  lead_id UUID NOT NULL REFERENCES leads(id) ON DELETE CASCADE,
  performed_by UUID REFERENCES profiles(id),
  type TEXT NOT NULL
    CHECK (type IN ('call', 'email', 'whatsapp', 'site_visit',
                    'note', 'status_change', 'score_update')),
  description TEXT NOT NULL,
  old_value TEXT,
  new_value TEXT,
  metadata JSONB DEFAULT '{}',
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

whatsapp_messages

```
CREATE TABLE whatsapp_messages (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  lead_id UUID REFERENCES leads(id) ON DELETE SET NULL,
  direction TEXT NOT NULL CHECK (direction IN ('inbound', 'outbound')),
  message_type TEXT DEFAULT 'text'
    CHECK (message_type IN ('text', 'template', 'image', 'document')),
  content TEXT,
  template_name TEXT,
  media_url TEXT,
  status TEXT DEFAULT 'sent'
    CHECK (status IN ('queued', 'sent', 'delivered', 'read', 'failed')),
  whatsapp_message_id TEXT,
  error_message TEXT,
```

```
  created_at TIMESTAMPTZ DEFAULT NOW()
);
```

subscriptions

```
CREATE TABLE subscriptions (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
  razorpay_subscription_id TEXT UNIQUE,
  razorpay_payment_id TEXT,
  plan TEXT NOT NULL CHECK (plan IN ('free', 'pro', 'business', 'enterprise')),
  status TEXT DEFAULT 'active'
    CHECK (status IN ('created', 'active', 'paused', 'cancelled', 'expired')),
  amount NUMERIC,
  currency TEXT DEFAULT 'INR',
  current_period_start TIMESTAMPTZ,
  current_period_end TIMESTAMPTZ,
  cancelled_at TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  updated_at TIMESTAMPTZ DEFAULT NOW()
);
```

Row Level Security (RLS) Policies

```
-- Enable RLS on all tables
ALTER TABLE organizations ENABLE ROW LEVEL SECURITY;
ALTER TABLE profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE listings ENABLE ROW LEVEL SECURITY;
ALTER TABLE leads ENABLE ROW LEVEL SECURITY;
ALTER TABLE lead_activities ENABLE ROW LEVEL SECURITY;
ALTER TABLE whatsapp_messages ENABLE ROW LEVEL SECURITY;
ALTER TABLE subscriptions ENABLE ROW LEVEL SECURITY;

-- Users can only see their organization's data
CREATE POLICY "org_isolation" ON listings
  FOR ALL USING (
    organization_id IN (
      SELECT organization_id FROM profiles WHERE id = auth.uid()
    )
  );
-- (Similar policies for leads, lead_activities, whatsapp_messages, subscriptions)
```

Indexes

```sql
CREATE INDEX idx_listings_org ON listings(organization_id);
CREATE INDEX idx_listings_status ON listings(status);
CREATE INDEX idx_listings_city ON listings(city);
CREATE INDEX idx_leads_org ON leads(organization_id);
CREATE INDEX idx_leads_status ON leads(status);
CREATE INDEX idx_leads_score ON leads(ai_score DESC);
CREATE INDEX idx_leads_followup ON leads(next_followup_at);
CREATE INDEX idx_lead_activities_lead ON lead_activities(lead_id);
CREATE INDEX idx_whatsapp_messages_lead ON whatsapp_messages(lead_id);
```

---

6. DESIGN SYSTEM

Color Palette

Primary Colors

Token	Hex	Usage
`primary-50`	`#EFF6FF`	Light blue backgrounds
`primary-100`	`#DBEAFE`	Hover backgrounds
`primary-500`	`#3B82F6`	Links, secondary buttons
`primary-600`	`#2563EB`	Primary buttons, active states
`primary-700`	`#1D4ED8`	Button hover
`primary-900`	`#1E3A8A`	Dark accents

Semantic Colors

Token	Hex	Usage
`success-500`	`#10B981`	Converted, active, success
`warning-500`	`#F59E0B`	Pending, scheduled, caution
`danger-500`	`#EF4444`	Lost, errors, delete actions
`info-500`	`#6366F1`	Information, AI-related

Neutral Colors

Token	Hex	Usage
`white`	`#FFFFFF`	Main background
`slate-50`	`#F8FAFC`	Sidebar, card backgrounds
`slate-100`	`#F1F5F9`	Input backgrounds, table rows
`slate-200`	`#E2E8F0`	Borders, dividers
`slate-400`	`#94A3B8`	Placeholder text
`slate-500`	`#64748B`	Secondary text, labels
`slate-700`	`#334155`	Body text
`slate-900`	`#0F172A`	Headings, primary text

Typography

Element	Font	Size	Weight	Line Height
Display	Inter	36px	800	1.2
H1	Inter	30px	700	1.3
H2	Inter	24px	600	1.35
H3	Inter	20px	600	1.4
H4	Inter	16px	600	1.4
Body	Inter	14px	400	1.5
Body Small	Inter	13px	400	1.5
Caption	Inter	12px	400	1.4
Button Large	Inter	16px	500	1
Button	Inter	14px	500	1
Button Small	Inter	12px	500	1

Spacing Scale

Token	Value
xs	4px
sm	8px
md	12px

lg	16px
xl	24px
2xl	32px
3xl	48px
4xl	64px

Border Radius

Token	Value	Usage
sm	4px	Badges, chips
md	6px	Buttons, inputs
lg	8px	Cards, modals
xl	12px	Large cards
full	9999px	Avatars, pills

Shadows

Token	Value	Usage
sm	0 1px 2px rgba(0,0,0,0.05)	Buttons, inputs
md	0 4px 6px rgba(0,0,0,0.07)	Cards, dropdowns
lg	0 10px 15px rgba(0,0,0,0.1)	Modals, popovers
xl	0 20px 25px rgba(0,0,0,0.1)	Dialogs

Component Specifications

Button Variants

Variant	Background	Text	Border	Hover
Primary	primary-600	white	none	primary-700
Secondary	white	slate-700	slate-200	slate-50
Outline	transparent	primary-600	primary-600	primary-50
Ghost	transparent	slate-600	none	slate-100
Danger	danger-500	white	none	danger-600

Button Sizes

Size	Height	Padding X	Font Size
sm	32px	12px	12px
md	36px	16px	14px
lg	44px	20px	16px

Input Specifications

State	Border	Background	Shadow
Default	slate-200	white	none
Focus	primary-500	white	ring primary-100
Error	danger-500	danger-50	ring danger-100
Disabled	slate-100	slate-50	none

Lead Status Badges

Status	Background	Text
New	blue-100	blue-700
Contacted	purple-100	purple-700
Interested	amber-100	amber-700
Site Visit	cyan-100	cyan-700
Negotiation	orange-100	orange-700
Converted	green-100	green-700
Lost	red-100	red-700

Listing Status Badges

Status	Background	Text
Draft	slate-100	slate-600
Active	green-100	green-700
Sold	blue-100	blue-700
Rented	purple-100	purple-700
Archived	slate-100	slate-500

Layout Specifications

Sidebar

- Width: 256px (expanded), 64px (collapsed)
- Background: white with right border (slate-200)
- Logo area height: 64px
- Nav item height: 40px
- Nav item padding: 8px 12px
- Active item: primary-50 background, primary-600 text
- Collapse breakpoint: 1024px (auto-collapse on tablet)

Topbar

- Height: 64px
- Background: white with bottom border (slate-200)
- Contains: Search bar, Notifications bell, User avatar dropdown

Content Area

- Max width: 1280px (centered)
- Padding: 24px (desktop), 16px (mobile)
- Page header height: ~80px (title + breadcrumb + actions)

Mobile Layout

- Sidebar hidden, replaced by hamburger menu
- Bottom tab navigation: Dashboard, Listings, Leads, Messages, More
- Tab bar height: 60px

Design Inspirations

- **Dashboard**: Linear.app (clean, minimal), Notion (sidebar navigation)
- **Tables**: Airtable (filter chips, views), Stripe Dashboard (clean data tables)
- **Forms**: Typeform (multi-step), Stripe (clean inputs)
- **Kanban**: Trello (drag-and-drop), Linear (status columns)
- **Cards**: Airbnb (property cards), Zillow (listing cards)
- **Overall**: Clean SaaS aesthetic with lots of whitespace

---

7. DIRECTORY STRUCTURE

```
propgenius/
├── .env.local                        # All environment variables
├── .env.example                      # Template for team members
├── .gitignore
├── .github/
│   └── workflows/
│       └── ci.yml                    # CI/CD pipeline
├── next.config.ts                    # Next.js configuration
├── package.json
├── tsconfig.json
```

```
├── postcss.config.mjs
│
├── public/
│   ├── logo.svg                      # PropGenius logo
│   ├── logo-icon.svg                 # Small icon version
│   ├── favicon.ico
│   └── og-image.png                  # Social sharing image
│
└── src/
    ├── middleware.ts                  # Auth guard + subscription check
    │
    ├── app/
    │   ├── layout.tsx                # Root layout (font, providers)
    │   ├── globals.css               # Tailwind + design tokens
    │   ├── loading.tsx               # Root loading state
    │   ├── not-found.tsx             # 404 page
    │   │
    │   ├── (auth)/
    │   │   ├── layout.tsx            # Centered auth layout
    │   │   ├── login/
    │   │   │   └── page.tsx          # Email/password + Google OAuth
    │   │   ├── signup/
    │   │   │   └── page.tsx          # Registration + org creation
    │   │   └── forgot-password/
    │   │       └── page.tsx          # Password reset
    │   │
    │   ├── (dashboard)/
    │   │   ├── layout.tsx            # Sidebar + Topbar wrapper
    │   │   ├── page.tsx              # Dashboard home
    │   │   │
    │   │   ├── listings/
    │   │   │   ├── page.tsx          # All listings (table view)
    │   │   │   ├── new/
    │   │   │   │   └── page.tsx      # Multi-step create form
    │   │   │   └── [id]/
    │   │   │       ├── page.tsx      # Listing detail view
    │   │   │       └── edit/
    │   │   │           └── page.tsx  # Edit listing
    │   │   │
    │   │   ├── leads/
    │   │   │   ├── page.tsx          # Leads (table + kanban toggle)
    │   │   │   ├── new/
    │   │   │   │   └── page.tsx      # Add lead form
    │   │   │   └── [id]/
    │   │   │       └── page.tsx      # Lead detail + timeline
    │   │   │
    │   │   ├── messages/
    │   │   │   └── page.tsx          # WhatsApp conversations
    │   │   │
```

```
│   │   ├── analytics/
│   │   │   └── page.tsx          # Charts and reports
│   │   │
│   │   └── settings/
│   │       ├── page.tsx          # General settings
│   │       ├── billing/
│   │       │   └── page.tsx      # Plan & subscription
│   │       ├── team/
│   │       │   └── page.tsx      # Team management
│   │       └── integrations/
│   │           └── page.tsx      # WhatsApp, platform setup
│   │
│   └── api/
│       ├── auth/
│       │   └── callback/
│       │       └── route.ts      # Supabase OAuth callback
│       ├── ai/
│       │   ├── generate-listing/
│       │   │   └── route.ts      # AI listing generation
│       │   ├── score-lead/
│       │   │   └── route.ts      # AI lead scoring
│       │   └── social-post/
│       │       └── route.ts      # AI social media post
│       ├── webhooks/
│       │   ├── razorpay/
│       │   │   └── route.ts      # Payment webhooks
│       │   └── whatsapp/
│       │       └── route.ts      # WhatsApp webhooks
│       └── leads/
│           └── import/
│               └── route.ts      # CSV bulk import
│
├── lib/
│   ├── constants.ts              # App config, enums, limits
│   ├── utils.ts                  # cn(), formatPrice(), formatDate()
│   ├── validations.ts            # Zod schemas for all forms/APIs
│   ├── gemini.ts                 # AI service class
│   ├── razorpay.ts               # Payment helpers
│   ├── whatsapp.ts               # WhatsApp API helpers
│   ├── resend.ts                 # Email service
│   └── supabase/
│       ├── client.ts             # Browser client
│       ├── server.ts             # Server client (cookies)
│       ├── middleware.ts          # Session refresh
│       └── admin.ts              # Service role client
│
├── types/
│   ├── listing.ts                # Property & listing types
│   ├── lead.ts                   # Lead & activity types
```

```
│   ├── user.ts                    # Profile & organization types
│   ├── message.ts                 # WhatsApp message types
│   ├── subscription.ts            # Billing & plan types
│   └── analytics.ts              # Chart data types
│
├── stores/
│   ├── auth-store.ts              # User session & profile
│   ├── sidebar-store.ts           # Sidebar collapsed state
│   └── notification-store.ts      # Toast notifications
│
├── hooks/
│   ├── use-auth.ts                # Auth actions
│   ├── use-listings.ts            # Listing CRUD + queries
│   ├── use-leads.ts               # Lead CRUD + queries
│   ├── use-messages.ts            # WhatsApp messages
│   ├── use-analytics.ts           # Dashboard data
│   ├── use-subscription.ts        # Plan & billing
│   └── use-debounce.ts            # Input debounce
│
└── components/
    ├── providers.tsx              # QueryClient + Auth + Toast
    ├── sidebar.tsx                # Collapsible sidebar nav
    ├── topbar.tsx                 # Search, notifications, avatar
    ├── mobile-nav.tsx             # Bottom tab bar (mobile)
    │
    ├── ui/                        # Design System Primitives
    │   ├── button.tsx
    │   ├── input.tsx
    │   ├── textarea.tsx
    │   ├── select.tsx
    │   ├── badge.tsx
    │   ├── modal.tsx
    │   ├── toast.tsx
    │   ├── avatar.tsx
    │   ├── card.tsx
    │   ├── dropdown.tsx
    │   ├── tabs.tsx
    │   ├── data-table.tsx
    │   ├── skeleton.tsx
    │   ├── empty-state.tsx
    │   ├── spinner.tsx
    │   ├── breadcrumb.tsx
    │   └── page-header.tsx
    │
    ├── listings/                  # Listing Feature Components
    │   ├── listing-card.tsx       # Property card with image
    │   ├── listing-form.tsx       # Multi-step form
    │   ├── listing-table.tsx      # Data table with filters
    │   ├── ai-generator.tsx       # AI panel with preview
```

```
│   ├── image-uploader.tsx     # Drag-drop image upload
│   ├── amenity-picker.tsx     # Checkbox grid
│   └── platform-publisher.tsx # Publish to 99acres etc.
│
├── leads/                     # Lead Feature Components
│   ├── lead-card.tsx          # Compact lead card
│   ├── lead-table.tsx         # Filterable data table
│   ├── lead-kanban.tsx        # Drag-drop board
│   ├── lead-form.tsx          # Add/edit lead form
│   ├── lead-timeline.tsx      # Activity history
│   ├── lead-score-badge.tsx   # AI score display
│   └── followup-scheduler.tsx # Date/time picker
│
├── messages/                  # Messaging Components
│   ├── chat-list.tsx          # Conversation list
│   ├── chat-window.tsx        # Message thread
│   ├── message-bubble.tsx     # Individual message
│   └── template-picker.tsx    # Quick reply templates
│
├── analytics/                 # Analytics Components
│   ├── stat-card.tsx          # KPI card
│   ├── revenue-chart.tsx      # Line/area chart
│   ├── leads-chart.tsx        # Bar chart
│   ├── conversion-funnel.tsx  # Funnel visualization
│   └── top-listings.tsx       # Top performing list
│
└── settings/                  # Settings Components
    ├── plan-card.tsx          # Subscription plan card
    ├── team-table.tsx         # Team members table
    └── integration-card.tsx   # Integration toggle card
```

---

8. FEATURE BREAKDOWN

8.1 Authentication & Onboarding

- Email/password signup and login via Supabase Auth
- Google OAuth integration
- Forgot password with email reset link
- **Onboarding Wizard** (shown on first login):
  - Step 1: Organization name and type
  - Step 2: Select plan (can skip to free)
  - Step 3: Create first listing (guided)

8.2 Dashboard

- **4 Stat Cards**: Active Listings, Total Leads, Conversions This Month, AI Listings Generated
- **Leads Over Time Chart**: Line chart showing last 30 days
- **Listings by Status**: Donut/bar chart
- **Recent Activity Feed**: Last 10 actions across leads and listings
- **Upcoming Follow-ups**: Next 5 scheduled follow-ups with quick actions
- **Quick Actions Bar**: "New Listing" and "Add Lead" buttons

8.3 AI Listing Generator (Core Feature)

Multi-Step Form:

- **Step 1 - Basics**: Property type (dropdown), Transaction type (sale/rent/lease), Price (with unit selector)
- **Step 2 - Location**: Address (text), City (autocomplete), State (dropdown), Pincode, Optional: Google Maps pin
- **Step 3 - Details**: BHK config, Area (sqft), Floor, Furnishing, Facing, Age, Parking, Amenities (checkbox grid: Gym, Pool, Club, Garden, Power Backup, Lift, Security, etc.)
- **Step 4 - Images**: Drag-and-drop upload (up to 20 images), Auto-thumbnail generation, Reorder by drag, Floor plan upload (optional)
- **Step 5 - AI Generation**: Click "Generate with AI" button, Show loading state with animation, Display generated content in editable cards:
  - Property Title
  - Full Description (150-200 words)
  - 5 Key Highlights (bullet points)
  - Social Media Post (Instagram/Facebook ready)
  - SEO Meta Title + Description
  - Edit any field before saving

AI Prompt Template:

```
You are a professional Indian real estate copywriter. Generate a compelling
property listing for the following property:

Property Type: {type}
Transaction: {transaction_type}
Price: ₹{price}
Location: {locality}, {city}, {state}
Configuration: {bedrooms} BHK, {bathrooms} Bath
Area: {area_sqft} sq.ft.
Floor: {floor_number} of {total_floors}
Furnishing: {furnishing}
Amenities: {amenities_list}

Generate a JSON response with:
{
  "title": "SEO-friendly title (max 80 chars)",
  "description": "Professional description (150-200 words)",
  "highlights": ["highlight1", "highlight2", "highlight3", "highlight4", "highlight5"],
```

```
  "social_post": "Instagram/Facebook post with emojis (max 300 chars)",
  "seo_title": "Meta title for search engines (max 60 chars)",
  "seo_description": "Meta description (max 160 chars)"
}

Guidelines:
- Use professional but approachable tone
- Mention nearby landmarks if area is well-known
- Highlight unique selling points
- Use Indian real estate terminology (carpet area, super built-up, etc.)
- Do NOT make up amenities or features not provided
```

8.4 CRM / Lead Management

Table View

- Columns: Name, Phone, Source, Interested In, AI Score, Status, Assigned To, Next Follow-up, Created
- Sorting: Click column headers to sort
- Filtering: By status, source, score range, assigned agent, date range
- Search: By name, phone, email
- Bulk actions: Assign to agent, Change status, Export CSV

Kanban Board

- Columns: New | Contacted | Interested | Site Visit | Negotiation | Converted | Lost
- Drag cards between columns to update status
- Cards show: Name, phone, score badge, interested property, days since last contact
- Color coding by AI score: Green (>70), Yellow (40-70), Red (<40)

Lead Detail Page

- Contact information card
- Interested property link
- AI Score with explanation
- Activity timeline (calls, emails, WhatsApp, notes, status changes)
- Add note / Log call / Schedule follow-up actions
- Quick WhatsApp button

AI Lead Scoring

```
Analyze this real estate lead and provide a score from 0-100:

Lead Info:
- Name: {name}
- Source: {source}
- Budget: ₹{budget_min} - ₹{budget_max}
- Interested in: {listing_title} (₹{listing_price})
- Preferred location: {preferred_location}
- Created: {days_ago} days ago
```

```
- Times contacted: {contact_count}
- Last response: {days_since_response} days ago
- Status: {current_status}

Scoring factors:
- Budget alignment with interested property (higher = better match)
- Response recency (responded recently = higher score)
- Source quality (referral > website > portal)
- Engagement level (multiple inquiries = higher)
- Timeline urgency (actively looking vs browsing)

Return JSON:
{
  "score": 0-100,
  "reason": "Brief explanation of the score",
  "recommended_action": "What the agent should do next"
}
```

8.5 WhatsApp Integration

- **Setup**: Connect WhatsApp Business number via Twilio
- **Templates**: Pre-approved message templates for:
  - New listing alert
  - Follow-up reminder
  - Site visit confirmation
  - Price update notification
  - Thank you after visit
- **Chat Interface**: WhatsApp-like UI within the app
- **Auto-logging**: All messages logged to lead activity timeline
- **Notifications**: Browser notifications for new messages

8.6 Analytics Dashboard

- **KPIs**: Total listings, Active leads, Conversion rate, Avg. response time
- **Charts**:
  - Leads over time (line chart, 30/60/90 days)
  - Listings by type (bar chart)
  - Lead source breakdown (pie chart)
  - Conversion funnel (funnel chart)
  - Agent performance (if team plan)
- **AI Insights**: Natural language insights generated by Gemini:
  - "Your 2BHK apartments in Whitefield get 3x more inquiries"
  - "Leads from referrals convert 40% better than portal leads"
  - "Response time to new leads averages 4 hours — try to reduce to under 1 hour"
- **Export**: Download reports as CSV

8.7 Settings

- **General**: Organization name, logo, address, contact info
- **Profile**: Name, avatar, phone, email, password change
- **Billing**: Current plan, usage metrics, upgrade/downgrade, payment history
- **Team** (Business plan): Invite agents, assign roles, deactivate accounts
- **Integrations**: WhatsApp Business setup, platform API keys

---

9. AI INTEGRATION DETAILS

Gemini AI Service Architecture

```
// lib/gemini.ts - Service class structure
class GeminiService {
  // Singleton pattern
  private model: GenerativeModel;

  // Methods:
  generateListing(propertyData) → ListingAIContent
  scoreLead(leadData) → LeadScore
  generateSocialPost(listing) → string
  generateInsights(analyticsData) → string[]
  regenerateField(field, context) → string
}
```

AI Usage Limits by Plan

Plan	Listing Generations	Lead Scores	Social Posts
Free	5/month	10/month	5/month
Pro	Unlimited	Unlimited	Unlimited
Business	Unlimited	Unlimited	Unlimited

AI Safety & Quality

- Input validation before sending to Gemini
- Output parsing with Zod schema validation
- Fallback to template-based generation if AI fails
- Content moderation (no discriminatory language)

- User can always edit AI-generated content before publishing

---

10. API ROUTES

Method	Route	Auth	Description
POST	`/api/ai/generate-listing`	Required	Generate AI listing content
POST	`/api/ai/score-lead`	Required	Score a lead with AI
POST	`/api/ai/social-post`	Required	Generate social media post
POST	`/api/webhooks/razorpay`	Webhook Secret	Handle payment events
POST	`/api/webhooks/whatsapp`	Webhook Verify	Handle incoming WhatsApp
POST	`/api/leads/import`	Required	Bulk CSV lead import
GET	`/api/auth/callback`	Public	Supabase OAuth callback

Note: CRUD operations for listings, leads, etc. are handled directly through Supabase client libraries (not custom API routes), leveraging RLS policies for security.

---

11. DEVELOPMENT PHASES

Phase 1: Foundation (Week 1)

Goal: Working auth, layout, and empty dashboard

Tasks:

- Initialize Next.js project with TypeScript + Tailwind CSS
- Set up Supabase project (create tables, enable RLS)
- Configure environment variables
- Build design system: globals.css with CSS variables, all UI primitives
- Implement auth flow: signup, login, forgot password (Supabase Auth)
- Create dashboard layout: sidebar + topbar + content area
- Build dashboard page with static stat cards
- Set up CI/CD (GitHub Actions)

Deliverables:

- User can sign up, log in, see dashboard shell
- Sidebar navigation works
- Responsive layout (desktop + mobile)

Phase 2: Listings + AI (Week 2-3)

Goal: Full listing CRUD with AI generation

Tasks:

- Create listing TypeScript types and Zod validations
- Build multi-step listing form (5 steps)
- Implement image upload to Supabase Storage
- Create Gemini AI service class
- Build AI listing generation API route
- Create AI generator panel (loading, preview, edit)
- Build listing table view (TanStack Table)
- Create listing detail page
- Create listing card component
- Add listing status management

Deliverables:

- Agent can create a listing with AI-generated content
- View all listings in sortable/filterable table
- View individual listing details
- Upload and manage property images

Phase 3: CRM / Leads (Week 3-4)

Goal: Full lead management with AI scoring

Tasks:

- Create lead types and Zod validations
- Build lead table view with TanStack Table
- Build Kanban board with @dnd-kit
- Create lead detail page with activity timeline
- Implement AI lead scoring API route
- Build follow-up scheduler (date/time picker)
- Implement CSV lead import
- Build add/edit lead form
- Create lead score badge component
- Add activity logging system

Deliverables:

- Agent can add leads manually or via CSV import
- View leads in table or Kanban view
- AI scores leads automatically
- Track all activities on lead timeline
- Schedule and manage follow-ups

Phase 4: Messaging + WhatsApp (Week 5)

Goal: WhatsApp integration for lead communication

Tasks:

- Set up Twilio WhatsApp sandbox (or Meta Cloud API)
- Create WhatsApp webhook endpoint
- Build message types and handlers
- Create chat list component
- Build chat window with message bubbles
- Implement template message sending
- Auto-log messages to lead activities
- Set up Resend for email notifications
- Build notification system

Deliverables:

- Send WhatsApp messages to leads from the app
- Receive and view incoming messages
- Use pre-built templates for common messages
- All conversations logged to lead timeline

Phase 5: Analytics + Billing (Week 6)

Goal: Razorpay subscriptions and analytics dashboard

Tasks:

- Create Razorpay subscription plans
- Build plan selection UI
- Implement Razorpay checkout flow
- Create payment webhook handler
- Enforce plan limits (listings, leads, features)
- Build analytics dashboard with Recharts
- Create stat cards, charts, funnel
- Implement AI insights generation
- Add export to CSV functionality
- Build billing settings page with payment history

Deliverables:

- Users can subscribe to Pro/Business plans
- Plan limits enforced throughout the app
- Analytics dashboard with charts and AI insights
- Billing management in settings

Phase 6: Polish + Launch (Week 7)

Goal: Production-ready application

Tasks:

- Responsive design audit (mobile, tablet, desktop)

- Error boundary and error handling
- Loading states and skeleton screens everywhere
- Empty states with helpful CTAs
- SEO optimization (meta tags, OG images)
- Performance optimization (lazy loading, image optimization)
- Create marketing landing page
- Write user documentation / help pages
- Deploy to Vercel (production)
- Configure custom domain
- Set up monitoring (Vercel Analytics)
- Final testing checklist

Deliverables:

- Polished, production-ready SaaS application
- Landing page for marketing
- Deployed on custom domain
- CI/CD pipeline running

---

12. ENVIRONMENT VARIABLES

```
# ============================================
# Supabase
# ============================================
NEXT_PUBLIC_SUPABASE_URL=https://xxxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGci...
SUPABASE_SERVICE_ROLE_KEY=eyJhbGci...     # Server-only! Never expose

# ============================================
# Google Gemini AI
# ============================================
GEMINI_API_KEY=AIzaSy...                  # Server-only (used in API routes)

# ============================================
# Razorpay Payments
# ============================================
NEXT_PUBLIC_RAZORPAY_KEY_ID=rzp_test_...   # Public (needed for checkout)
RAZORPAY_KEY_SECRET=xxxxx                  # Server-only
RAZORPAY_WEBHOOK_SECRET=xxxxx              # For webhook verification

# ============================================
# WhatsApp (Twilio)
# ============================================
TWILIO_ACCOUNT_SID=ACxxxxx
TWILIO_AUTH_TOKEN=xxxxx
```

```
TWILIO_WHATSAPP_NUMBER=+14155238886        # Twilio sandbox number

# ============================================
# Email (Resend)
# ============================================
RESEND_API_KEY=re_xxxxx

# ============================================
# Google Maps (Optional)
# ============================================
NEXT_PUBLIC_GOOGLE_MAPS_KEY=AIzaSy...

# ============================================
# App Configuration
# ============================================
NEXT_PUBLIC_APP_URL=http://localhost:3000   # Change for production
```

---

13. DEPLOYMENT & DEVOPS

Vercel Deployment

- Connect GitHub repo to Vercel
- Set root directory to `/` (since it's a standalone project)
- Add all environment variables in Vercel dashboard
- Configure custom domain (e.g., propgenius.in)
- Enable Vercel Analytics and Web Vitals

CI/CD Pipeline (GitHub Actions)

```yaml
name: CI
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
```

```
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npm run type-check
      - run: npm run build
```

Branch Strategy

- `main` — Production (auto-deploys to Vercel)
- `develop` — Staging
- `feature/*` — Feature branches (PR to develop)
- `fix/*` — Bug fix branches

---

14. TESTING STRATEGY

Manual Testing Checklist

1. Auth: Signup → Login → Logout → Forgot Password → Reset
2. Listing: Create (all steps) → AI Generate → Edit → Save → View → Delete
3. Lead: Add → AI Score → Move Kanban → Log Activity → Schedule Follow-up
4. WhatsApp: Send Template → Receive Reply → View Conversation
5. Billing: Subscribe → Verify Limits → Upgrade → Cancel
6. Analytics: Verify counts → Charts render → Export CSV
7. Mobile: All pages responsive → Bottom nav works → Forms usable
8. Edge Cases: Empty states → Error states → Loading states → Offline behavior

Build Verification

- `npm run lint` — No ESLint errors
- `npm run type-check` — No TypeScript errors
- `npm run build` — Successful production build
- Lighthouse score > 90 on all pages

---

15. SECURITY CONSIDERATIONS

- **Row Level Security (RLS)**: All Supabase tables have RLS policies ensuring org-level data isolation
- **Server-only secrets**: Razorpay secret, Supabase service role key, Twilio auth token never exposed to client

- **Webhook verification**: Razorpay and WhatsApp webhooks verified with signatures
- **Input validation**: All user inputs validated with Zod schemas (both client and server)
- **Rate limiting**: AI generation endpoints rate-limited per plan
- **CSRF protection**: Next.js built-in CSRF protection on API routes
- **XSS prevention**: React's built-in escaping + sanitize user-generated content
- **Image upload**: File type validation, size limits (5MB/image, 20 images max)
- **Auth tokens**: Stored in HTTP-only cookies (Supabase SSR), not localStorage

---

16. FUTURE ROADMAP

v1.1 (Month 2)

- Virtual tour integration (Matterport)
- Automated property matching (lead preferences → matching listings)
- SMS integration (for non-WhatsApp users)

v1.2 (Month 3)

- Mobile app (React Native, share code with web)
- Voice-to-listing (speak property details, AI transcribes)
- Multi-language support (Hindi, Marathi, Tamil, Telugu)

v2.0 (Month 6)

- Marketplace: Public property listing portal
- Agent profile pages (shareable)
- Advanced analytics with predictive insights
- API for third-party integrations
- White-label solution for large brokerages

v3.0 (Year 1)

- Document management (agreements, legal docs)
- Payment collection for rent/deposits
- Tenant management module
- Property valuation AI (using historical data)

APPENDIX: QUICK REFERENCE

Key Commands

```
# Development
npm run dev           # Start dev server (localhost:3000)
npm run build         # Production build
npm run lint          # Run ESLint
npm run type-check    # Run TypeScript check

# Database
# Run SQL migrations in Supabase Dashboard → SQL Editor

# Deployment
git push origin main  # Auto-deploys to Vercel
```

Key URLs

- App: https://propgenius.in (production)
- Supabase Dashboard: https://supabase.com/dashboard
- Razorpay Dashboard: https://dashboard.razorpay.com
- Vercel Dashboard: https://vercel.com/dashboard
- Twilio Console: https://console.twilio.com

Document prepared for PropGenius AI project Tech Lead: Tarun Date: February 2026